

Distributed Systems

Contents

What changes when a system is distributed

1. [What a distributed system is](#)
2. [The fallacies of distributed computing](#)
3. [Partial failure](#)

Time and ordering

4. [Clocks and why they lie](#)
5. [Happens-before and logical clocks](#)

Consistency

6. [Consistency models](#)
7. [CAP and PACELC](#)

Replication, partitioning and agreement

8. [Replication topologies](#)
9. [Partitioning and sharding](#)
10. [Quorums](#)
11. [Consensus and leader election](#)

Messaging

12. [Delivery semantics](#)
13. [Ordering, partitions and consumer groups](#)
14. [Transactions across services](#)

Running it in production

15. [Service discovery and load balancing](#)
16. [Failure detection](#)
17. [Cascading failure and backpressure](#)
18. [Distributed locks](#)
19. [Tracing a request across services](#)

Reference

- 20. [Interview questions](#)
- 21. [Common mistakes](#)
- 22. [Quick reference](#)

1. What a distributed system is

A distributed system is a set of computers that appears to its users as one system, where the parts communicate only by passing messages over a network. Leslie Lamport's description is the one people quote, and it is accurate: a distributed system is one in which the failure of a computer nobody knew existed can make their own computer unusable.

Systems are distributed for four reasons, and each one is paid for in complexity:

- **Scale**, when the work no longer fits on one machine.
- **Fault tolerance**, so one machine failing does not stop the service.
- **Latency**, by putting data near the users who read it.
- **Organisation**, so separate teams can deploy separately.

What changes is not the amount of code but the kind of reasoning. In one process, either the process works or it dies. Across a network, **some parts work and some do not, and often we cannot tell which**. Almost everything in this document follows from that one fact.

2. The fallacies of distributed computing

Eight assumptions, collected at Sun Microsystems by Peter Deutsch and others, that everyone makes by accident when writing code that crosses a network:

Fallacy	What it means in practice
The network is reliable	Messages are lost, duplicated and delayed. Every call needs a timeout and a failure path
Latency is zero	A local call is nanoseconds, a remote one is milliseconds. Loops that call remotely become disasters
Bandwidth is infinite	Payload size matters, and chatty interfaces cost more than the work they do
The network is secure	Traffic is interceptable. Encrypt in transit and authenticate between services
Topology does not change	Instances come and go constantly. Addresses cannot be hardcoded
There is one administrator	Different teams own different pieces, on different release cycles
Transport cost is zero	Serialisation, bandwidth and cross-zone traffic all appear on a bill
The network is homogeneous	Protocols, versions and encodings differ, and have to be negotiated

The first two do most of the damage. A method that looks like an ordinary call but crosses a network can fail in ways a local call cannot, and takes six orders of magnitude longer when it succeeds.

3. Partial failure

In a single process, failure is total: the process runs or it does not. In a distributed system, failure is **partial**, and the caller usually cannot tell which kind it hit.

When a request times out, three things may have happened:

1. The request never arrived.
2. It arrived, the work completed, and the response was lost.
3. It is still being processed, and will complete after the timeout.

Nothing in the timeout distinguishes them. That single ambiguity is why retries need idempotency, why the API note has a section on idempotency keys, and why "did the payment go through" is a hard question rather than a lookup.

It also sets a limit on delivery. A sender that receives no acknowledgement must choose between resending, which risks a duplicate, and not resending, which risks a loss. No protocol removes that choice, so **exactly-once delivery cannot be guaranteed by the communication layer alone**, which section 12 picks up.

A related result is the **Two Generals Problem**, which concerns agreement rather than delivery. Two generals must agree to attack together and can only communicate by messengers who may be captured. No number of acknowledgements makes both certain the other will attack, because the last message sent is always unconfirmed. Both results come from the same root: over an unreliable channel, a sender never gains certainty about the receiver's state.

Most systems assume **crash-stop** or **crash-recovery** failures, where a node stops or restarts but behaves correctly while running. **Byzantine** faults, where a node behaves arbitrarily or dishonestly, need much more expensive algorithms. They matter wherever nodes may be compromised or are not mutually trusted, which includes systems spanning administrative boundaries, safety-critical systems, and public blockchains. Inside one organisation the usual assumption is non-Byzantine, and saying so is a reasonable thing to state out loud in a design discussion.

4. Clocks and why they lie

Two kinds of clock, and using the wrong one is a common bug.

A **wall clock**, or time-of-day clock, reports human time. It is synchronised by NTP, Network Time Protocol, and can therefore jump, including backwards, when a correction lands. It is right for timestamps a person will read and wrong for measuring elapsed time.

A **monotonic clock** only moves forward, at a rate the machine controls. It has no meaning across machines and no relationship to human time. It is right for durations and timeouts.

Java exposes both, and the distinction is worth knowing precisely:

```
// Wrong. currentTimeMillis is a wall clock, so an NTP correction
// during the operation can make this negative or wildly large.
long start = System.currentTimeMillis();
doWork();
long elapsed = System.currentTimeMillis() - start;

// Right. nanoTime is monotonic and immune to clock adjustment.
long start = System.nanoTime();
doWork();
Duration elapsed = Duration.ofNanos(System.nanoTime() - start);
```

`System.nanoTime()` has no relationship to any calendar, so comparing values from two different machines is meaningless.

Clock skew is the difference between two machines' wall clocks. How large it gets depends on the network, the hardware, the operating system and how well synchronisation is working, and it grows without warning when synchronisation degrades quietly. The consequence that matters: **ordering events across machines by wall-clock timestamp is unreliable**. "Last write wins" conflict resolution by timestamp is a mechanism for silently discarding writes, because the write carrying the later timestamp may not be the one that happened later.

Google Spanner is worth naming as the exception that proves the point. It uses atomic clocks and satellite receivers, and its TrueTime interface returns an interval rather than an instant, admitting its own uncertainty. Transactions then wait out that uncertainty before committing, which buys real ordering at the cost of hardware most systems do not have.

5. Happens-before and logical clocks

Since physical clocks cannot order events across machines, ordering has to be derived from causality instead.

Lamport defined the **happens-before** relation, written `a → b`:

- If `a` and `b` are in the same process and `a` came first, then `a → b`.
- If `a` is sending a message and `b` is receiving that message, then `a → b`.
- It is transitive: if `a → b` and `b → c`, then `a → c`.

Events with no happens-before relation in either direction are **concurrent**. That is a precise statement about information flow, not about physical time: two concurrent events may occur hours apart, as long as neither could have influenced the other.

Lamport timestamps implement it cheaply. Each node keeps a counter, increments it on every event, and attaches it to outgoing messages. On receiving a message the node sets its counter to `max(local, received) + 1`. The result gives a total order consistent with causality, but the converse does not hold: `a < b` in timestamps does not prove `a → b`, so concurrency cannot be detected.

Vector clocks fix that by keeping one counter per node, so comparing two vectors shows whether one strictly precedes the other or whether they are concurrent. Dynamo-style stores use them to detect conflicting writes rather than silently overwriting one. The cost is that a vector grows with the number of nodes that have ever written, so it needs pruning in large clusters.

6. Consistency models

A consistency model is a contract about what a read may return. They form a spectrum, and it is worth being able to place four of them.

Linearizability, sometimes called strong consistency or atomic consistency. Every operation appears to take effect instantaneously at some point between its start and its completion, and that order respects real time. The system behaves as though there were one copy of the data. It is the strongest of these models, and in a replicated system it is usually the most expensive, because keeping replicas in agreement on every operation adds latency and reduces availability during failures. A single node is trivially linearizable; the cost appears once the data is replicated.

Sequential consistency. Every node sees operations in the same order, but that order need not match real time. A write completed before another began may still be ordered after it.

Causal consistency. Operations related by happens-before are seen in that order by everyone. Concurrent operations may be seen in different orders by different nodes. This is often the practical middle ground, because it prevents the visibly wrong outcomes, such as seeing a reply before the message it answers, without the coordination cost of ordering everything.

Eventual consistency. If writes stop, replicas converge on the same value. It says nothing about how long that takes, or what a read returns meanwhile. It is a real guarantee, and a minimal one, so treating it as a synonym for "weak" understates what convergence requires and overstates what it promises.

Underneath these sit **session guarantees**, which are usually what an application actually needs:

Guarantee	Promise
Read your writes	A client always sees its own writes afterwards
Monotonic reads	A client never sees time go backwards across reads
Monotonic writes	A client's writes are applied in the order it issued them
Writes follow reads	A write is ordered after any write the client already read

The read-your-writes case is the one that produces bug reports, since a user who writes to a leader and then reads from a lagging replica sees their change vanish. The database note covers that failure from the replication side.

7. CAP and PACELC

CAP states that when a network **P**artition occurs, a system must choose between **C**onsistency, meaning linearizability, and **A**vailability, meaning every request to a non-failing node eventually receives a response.

Three misreadings worth being able to correct:

- **It is not "pick two of three" as a design-time menu.** Partitions are a property of networks, not a choice. In a distributed system partition tolerance is mandatory, so the real decision is what happens during a partition.
- **The C is linearizability**, not the C in ACID, which means declared constraints still hold. The database note makes the same distinction from the other direction.
- **It describes behaviour during a partition**, so no system permanently belongs to one category. A system can be strongly consistent normally and choose availability under partition, or the reverse.

PACELC extends it, and is the more useful framing day to day. If there is a **P**artition, choose **A**vailability or **C**onsistency; Else, in normal operation, choose **L**atency or **C**onsistency. Partitions are rare. The latency-against-consistency trade runs constantly, because every synchronous replication acknowledgement is latency bought for consistency.

8. Replication topologies

Replication keeps copies of the **same** data on several nodes, for durability, read capacity and availability.

Topology	Writes go to	Strengths	Costs
Single leader	One node, replicated onward	No write conflicts, simple to reason about	Leader is a write bottleneck and a failover event
Multi leader	Several nodes, replicated between them	Works across regions and offline clients	Write conflicts are now possible and must be resolved
Leaderless	Several replicas directly, by the client	No failover step, tolerates node loss well	Conflict resolution and repair become the caller's problem

Single leader is what most relational databases do, and the database note covers replication lag and failover from that angle.

Multi leader and leaderless designs both create the same new problem: two writes to the same key with no ordering between them. Common approaches to resolving that, and what each costs:

- **Last write wins**, by timestamp. Simple, and it discards data, for the clock reasons in section 4.

- **Version vectors**, which detect the conflict and hand both versions to the application to merge.
- **CRDTs, Conflict-free Replicated Data Types**, which are structured so that concurrent updates merge deterministically with no coordination. Counters, sets and some text structures have useful CRDT forms, and the constraint is that the data has to fit one of those shapes.
- **Application-level merge**, where domain knowledge decides, such as taking the union of two shopping carts. The most flexible and the most work.

9. Partitioning and sharding

Replication copies the same data to several nodes. **Partitioning**, also called sharding, splits *different* data across nodes so each one owns a subset. They are independent choices and most systems use both, so a typical arrangement is a dataset split into partitions with each partition replicated three times.

Three ways to decide which node owns a key:

Range partitioning keeps keys in sorted order and gives each node a contiguous range. Range scans stay efficient, because neighbouring keys live together. The risk is a hot partition when the key is sequential: partitioning by timestamp sends every new write to whichever node holds the newest range, while the others idle.

Hash partitioning applies a hash function to the key and uses that to choose a node. Distribution is even and sequential keys stop being a problem. The cost is that ordering is destroyed, so a range scan has to visit every partition.

Consistent hashing solves the problem that plain `hash(key) mod N` creates: changing `N` remaps almost every key, so adding one node moves nearly all the data. Consistent hashing places both nodes and keys on a ring, and a key belongs to the next node clockwise. Adding a node then moves only the keys between it and its neighbour, roughly a `1/N` share rather than everything. Virtual nodes, where each physical node appears at many points on the ring, smooth out the unevenness. Dynamo-style stores such as Cassandra are built on this.

Three problems that follow from any of them:

- **Hot partitions.** A single popular key, such as one celebrity account, cannot be split by any hashing scheme, because it is one key. The workarounds are application-level: add a random suffix to spread one logical key across several physical ones and merge on read, or handle that key differently.
- **Rebalancing.** Nodes are added and removed, and partitions have to move. The common approach is to create many more partitions than nodes at the start and reassign whole partitions rather than recompute ownership per key, which is what Kafka and Elasticsearch do.
- **Cross-partition operations.** A query that does not filter on the partition key has to visit every partition and combine the results, which is scatter-gather and gets slower as the cluster grows. Transactions spanning partitions need the machinery in section 14.

The partition key is therefore the decision that is hardest to reverse, because changing it means moving everything.

A Kafka partition is not a database shard

Both split data across nodes, and they answer different questions. A database shard exists so the dataset fits and the load spreads, and any node can serve any query for the rows it owns. A Kafka partition is an ordered log that exists to provide ordering within a key and parallelism across keys, and it is consumed by at most one member of a consumer group at a time. The shared word hides a different purpose.

10. Quorums

In a leaderless or quorum-based system with **N** replicas, a write waits for **W** acknowledgements and a read collects **R** responses.

When **W + R > N**, the set of replicas acknowledging a write and the set answering a read must overlap by at least one node, so under the usual quorum assumptions a read reaches a replica that accepted the newest write.

$N = 3, W = 2, R = 2$	→ $2 + 2 > 3$, overlap guaranteed
$N = 3, W = 3, R = 1$	→ slow writes, fast reads
$N = 3, W = 1, R = 3$	→ fast writes, slow reads
$N = 3, W = 1, R = 1$	→ no overlap, so a read may miss the latest write

Two qualifications keep this honest. Overlap means the read *encounters* a replica holding the newest value; the client still needs a version to work out which of the returned values that is. And quorum overlap on its own is not linearizability, since concurrent operations can still be observed in different orders without additional machinery such as read repair and enforced ordering.

Sloppy quorums weaken it deliberately. When the usual replicas are unreachable, writes go to whichever nodes are available, with **hinted handoff** delivering them later. Availability improves and the overlap guarantee no longer holds during the disruption.

11. Consensus and leader election

Consensus is getting a group of nodes to agree on one value despite failures and message loss. It underpins leader election, replicated logs, distributed configuration and lock services.

Atomic commit is a related but distinct problem. Consensus asks a group to agree on a value and can proceed with a majority. Atomic commit requires *every* participant to agree to commit, so one participant failing can block it. Two-phase commit is an atomic commit protocol rather than a consensus algorithm, and that difference is part of why it blocks where Raft does not.

Two results are worth naming.

FLP impossibility, from Fischer, Lynch and Paterson: in an asynchronous system where even one node may fail, no deterministic algorithm can guarantee that consensus terminates. Real systems live with this by giving up guaranteed termination rather than correctness. They use timeouts and randomised delays, so agreement is reached in practice while the theoretical worst case remains unbounded.

Majority quorums. For majority-based consensus under crash failures, a cluster of $2f + 1$ nodes tolerates f failures, because any two majorities overlap. That is why etcd and ZooKeeper clusters are sized at three or five rather than four: four nodes still tolerate only one failure and cost more. Byzantine fault tolerance needs stronger assumptions, commonly $3f + 1$ nodes to tolerate f dishonest ones, which is one reason it stays confined to environments where nodes cannot be trusted.

Raft is the algorithm to be able to describe, since it was designed to be understandable where Paxos was not. Time is divided into **terms**, each with at most one leader. A node that hears nothing from a leader becomes a candidate, increments the term and requests votes; a candidate winning a majority becomes leader. The leader takes all writes, appends them to its log and replicates to followers, and an entry becomes committed once Raft's commit conditions are met, which normally means replication to a majority. The full rule also involves the leader's current term, which is what stops a new leader drawing wrong conclusions about entries left over from an old one.

Split brain is the failure all of this exists to prevent:

```
Network partition
  ↓
Two nodes each believe they are authoritative
  ↓
Both accept writes
  ↓
State diverges, with no record of which side is correct
```

Requiring a majority is what prevents it, because two disjoint majorities cannot exist in one cluster, so at most one side of a partition can elect a leader. Terms make a stale leader detectable, since a node seeing a higher term stands down. Where an external resource is involved, majorities and terms are not enough on their own, and **fencing tokens** in section 18 are what close the gap.

12. Delivery semantics

Semantic	Behaviour	Cost
At most once	Send, do not retry	Messages can be lost
At least once	Retry until acknowledged	Messages can be duplicated
Exactly once	Each message takes effect once	See below

The precise answer, which is a strong thing to have ready: **exactly-once delivery is not achievable over an unreliable network**. A sender that receives no acknowledgement must either resend, risking a duplicate, or not, risking a loss, and no protocol removes that choice.

What systems do provide is **effectively-once processing**: at-least-once delivery combined with either idempotent handling or an atomic commit that ties the side effect to the acknowledgement. Kafka's exactly-once semantics are this, not an exception to it. An idempotent producer removes duplicates caused by producer retries, and transactions make the consume, process and produce cycle atomic **within Kafka**. A side effect outside that boundary, such as charging a card or calling another service, is still the application's problem to make idempotent.

So the practical rule is to design consumers to be idempotent and stop trying to make delivery perfect. The API note's idempotency key section is the same idea applied to HTTP.

13. Ordering, partitions and consumer groups

Kafka is the common example, and its model answers several interview questions at once.

A **topic** is split into **partitions**. **Ordering is guaranteed within a partition, and not across a topic**. A message's key decides its partition, so all events for one key land in one partition and are ordered relative to each other. Choosing the key is therefore the ordering decision: keying order events by order identifier keeps each order's events in sequence, while saying nothing about the order of events for two different orders.

A **consumer group** shares the work, with each partition assigned to at most one consumer in the group. Parallelism is therefore capped by partition count: twenty consumers on a ten-partition topic leaves ten idle.

Where the **offset** is committed decides the delivery semantic:

```
@KafkaListener(topics = "orders", groupId = "billing")
public void handle(ConsumerRecord<String, OrderEvent> record, Acknowledgment ack) {
    // Process first, acknowledge after. A crash before the ack means the
    // message is redelivered, so this handler has to be idempotent.
    billing.process(record.value());
    ack.acknowledge();
}
```

Committing after processing gives at-least-once, with redelivery on a crash. Committing before processing gives at-most-once, losing the message instead. There is no third option that avoids both, which is section 12 restated.

Two operational details worth knowing. A **rebalance** reassigns partitions when a consumer joins or leaves, and processing pauses while it happens, so a consumer that takes too long between polls gets evicted and triggers another one. And **consumer lag**, the distance between the latest offset and the committed offset, is the single most useful metric on a queue, because it shows whether consumers are keeping up.

14. Transactions across services

Two services with separate databases have no shared transaction. Writing to a database and publishing an event is the **dual write problem**: two systems, no atomicity, and a crash between them leaves the two disagreeing.

Three answers, covered in more depth in the ACID note:

Two-phase commit has a coordinator ask every participant to prepare, then tell them all to commit. It works, and it is rare, because participants hold locks while waiting and a coordinator failing between phases leaves them stuck.

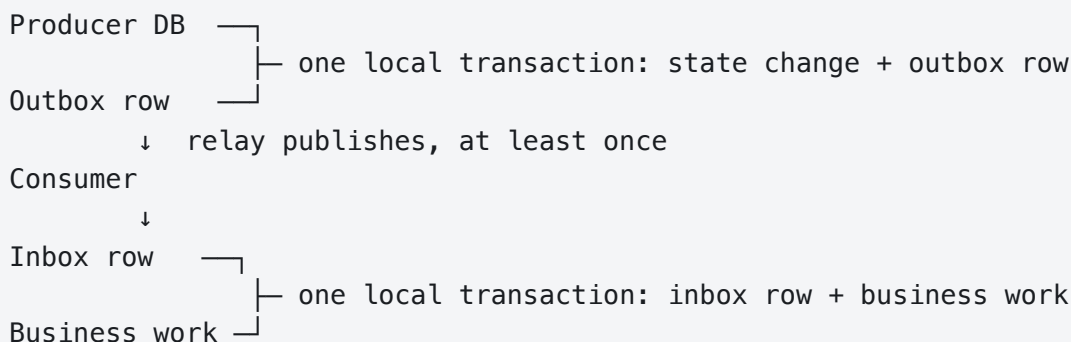
Saga breaks the work into local transactions, each with a compensating transaction that undoes it. There is no isolation between the steps, so intermediate states are visible, and recovery is a design problem rather than a database feature.

Transactional outbox is the one to name first for the dual write case. The state change and a row describing the event are written in the *same* local transaction, and a separate relay publishes from the outbox afterwards. Local atomicity now covers both.

The outbox fixes the producer. The **inbox**, or idempotent consumer, fixes the other end, and the pair is what makes at-least-once delivery safe end to end:

```
@Transactional
public void handle(OrderEvent event) {
    // Insert against a unique constraint on the message identifier.
    // Returns false when the row already exists, so this is a redelivery.
    if (!inbox.recordIfNew(event.messageId())) {
        return;
    }
    billing.charge(event);    // same transaction as the inbox row
}
```

Because the inbox row and the business work commit together, a redelivered message finds its identifier already recorded and does nothing. Put end to end:



Neither end needs a distributed transaction, and duplicates between them become harmless once the consumer enforces idempotency, which is what the inbox row is doing.

15. Service discovery and load balancing

Instances come and go constantly, which is the fallacy about topology from section 2. Addresses therefore cannot be configuration, and something has to answer "where is a healthy instance of this service".

A **service registry** holds that answer. Instances register when they start, deregister when they stop, and are removed when health checks fail. Consul, etcd and Eureka do this directly, and in Kubernetes the same job is done by Services and cluster DNS.

Two arrangements, and the trade is where the logic lives:

	Client-side discovery	Server-side discovery
Who chooses the instance	The caller, after querying the registry	A load balancer or proxy in front
Network hops	One	Two
Client complexity	Every client, in every language, needs the logic	Clients call one address
Where it appears	Eureka with Spring Cloud LoadBalancer, gRPC name resolvers	Kubernetes Services, a gateway, a service mesh sidecar

Once an instance has to be chosen, the strategy matters:

Strategy	Behaviour
Round robin	Even rotation. Ignores how busy each instance is
Least connections	Sends to the least busy, which suits uneven request costs
Weighted	Proportional to declared capacity, useful with mixed instance sizes
Consistent hashing	The same key always reaches the same instance, which helps local caching
Health or latency aware	Avoids instances that are slow or failing, rather than only those that are down

Health checks are what connect this to failure detection. The distinction from the API note applies here too: **readiness** decides whether an instance should receive traffic, and **liveness** decides whether it should be restarted. Conflating them restarts healthy processes during a brief dependency outage, when the correct response was to stop routing to them and wait.

16. Failure detection

A **slow node** and a **dead node** are **indistinguishable from the outside**. Every failure detector is therefore a guess, and the timeout is where the guess is encoded.

Heartbeats are the usual mechanism: nodes send periodic signals, and a node missing several is suspected. The threshold is a direct trade. Too short produces false positives, so healthy nodes are evicted, work is reassigned, and load rises, which causes more false positives. Too long means real failures go unnoticed and requests keep being routed into a hole.

A **phi accrual failure detector**, used in Cassandra and Akka, improves on a boolean answer by tracking the distribution of past heartbeat intervals and outputting a suspicion level. A caller decides what level justifies action, so a network that is merely slow does not immediately look dead.

Two related mechanisms are worth naming. A **lease** is permission held for a bounded time, so a node that cannot renew loses its role automatically, without anyone deciding it is dead. And **fencing**, in section 18, is what stops a node that was wrongly declared dead from doing damage when it returns.

17. Cascading failure and backpressure

A cascading failure is one component's problem becoming the whole system's. The usual sequence: a dependency slows, callers hold threads waiting, thread pools fill, callers start failing, their callers retry, load rises, and the original problem gets worse.

Three mechanisms keep it contained.

Backpressure signals upstream to slow down rather than accepting work that cannot be done. Bounded queues are the simplest form: a queue that rejects when full pushes the problem to a place that can decide about it, while an unbounded queue turns overload into growing latency and eventually memory exhaustion.

Load shedding rejects some requests early to keep the rest healthy. Failing ten percent of requests quickly is better than failing all of them slowly, and health-based shedding at the edge is more effective than every service deciding independently.

Isolation stops one dependency consuming everything. The API note covers circuit breakers, which stop calling a failing dependency, and bulkheads, which cap concurrent calls per dependency. Retries need backoff and jitter for the same reason, since synchronised retries are what turn a brief outage into a sustained one.

Little's Law is worth knowing as the arithmetic behind this: $L = \lambda W$, where the number of requests in the system equals arrival rate times average time in the system. If latency doubles at constant arrival rate, concurrency doubles, and that is why a small latency increase can exhaust a fixed-size pool.

A **metastable failure** is the state where the system stays down after the original trigger is gone, because the retry load it generated is now sufficient to keep it down. Recovery usually needs load to be shed deliberately rather than waiting.

18. Distributed locks

A distributed lock is harder than it looks, and explaining why is a strong answer.

The naive version uses an expiring key:

```
// Acquire only if absent, with a lease so a crash cannot hold it forever.
Boolean acquired = redis.opsForValue()
    .setIfAbsent("lock:order:42", nodeId, Duration.ofSeconds(10));
```

The flaw is not the code. It is that **a process can pause for longer than its lease**. A stop-the-world garbage collection pause, a hypervisor suspending the virtual machine, or a long input-output stall can freeze a JVM for seconds. During that pause the lease expires and another node acquires the lock legitimately. The first node then resumes, still believing it holds the lock, and writes. Two writers, no error anywhere.

Timeouts cannot fix this, because the paused process has no way to know time passed.

Fencing tokens are the mechanism that does. The lock service issues a monotonically increasing number with each grant, and the protected resource records the highest token it has seen and rejects anything lower:

```
long token = lockService.acquire("order:42");    // 33, then 34 for the next holder
storage.write(data, token);                     // rejected if token < last seen
```

The resumed node arrives with token 33 after the storage has already accepted 34, so its write is refused. The correctness now lives in the resource rather than in anyone's belief about the lock. That last part is the condition, not a detail: **a fencing token only helps if the protected resource actually validates it**. A token nobody checks is a number.

This is the key limitation behind debates about Redis-based distributed locks such as Redlock. The summary worth carrying: for efficiency, where a duplicate is merely wasteful, a lease is fine, and for correctness, where a duplicate is unacceptable, a lock without an enforced fencing token is not sufficient.

The better answer is usually to avoid needing one. Idempotent operations make duplicate execution harmless. Optimistic concurrency with a version column, from the database note, detects conflicts without any lock. Partitioning work so one key is only ever handled by one consumer, as Kafka does, removes the contention rather than managing it.

19. Tracing a request across services

One request touches five services and takes four seconds. Logs alone cannot say where the time went, because each service logged independently with nothing tying the lines together.

Distributed tracing supplies the missing identifier:

- A **trace identifier** names the whole end-to-end operation and travels with it across every process boundary. For an incoming request it is usually created at the edge, though a trace can equally begin at a scheduled job, a background task or a message consumer.
- A **span identifier** names one operation within it, such as a single service call or database query. Spans record a parent, so the trace forms a tree.
- **Propagation** carries the trace across process boundaries in headers. The W3C Trace Context standard defines `traceparent` for this, which is what makes tools from different vendors interoperate.

```

trace 0af7651916cd43dd
├─ span: POST /orders          420 ms
│   └─ span: inventory.reserve 18 ms
│       └─ span: payments.charge 380 ms ← the answer
│           └─ span: INSERT orders 6 ms

```

The three signals answer different questions, and separating them is worth a mark: **metrics** say something is wrong, **traces** say where, and **logs** say what happened there. They join through the identifier, which is why putting it into the logging context matters:

```

// Every log line inside this block carries the trace identifier, so logs
// for one request can be pulled together and matched against the trace.
try (var ignored = MDC.putCloseable("traceId", traceId)) {
    orders.create(command);
}

```

In Java the usual arrangement is Micrometer Tracing over OpenTelemetry, which instruments the common clients so propagation happens without hand-written header code.

One practical constraint: recording every trace is expensive at volume, so systems sample. **Head-based** sampling decides at the start of a request and is cheap but keeps traces at random. **Tail-based** sampling decides after the whole trace is collected, so it can keep the slow and failed ones, at the cost of buffering everything until the decision.

20. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What makes distributed systems hard?"

Partial failure, mostly. In a single process either it works or it dies, but across a network some parts work and some do not, and the caller usually cannot tell which. When a request times out there are three possibilities: it never arrived, it arrived and completed and the response was lost, or it is still running and will finish after the timeout. Nothing in the timeout distinguishes them, so every retry has to assume the work may already have happened, which is why idempotency comes up in every distributed design. The same uncertainty is why exactly-once delivery is not achievable, since a sender with no acknowledgement has to choose between resending and risking a duplicate, or not resending and risking a loss.

"Is exactly-once delivery possible?"

Not as delivery, no. Over an unreliable network the sender that gets no acknowledgement has to choose between resending, which risks a duplicate, and not resending, which risks a loss, and no protocol removes that choice. What is achievable is effectively-once processing, which is at-least-once delivery plus either idempotent handling or an atomic commit tying the side effect to the acknowledgement. Kafka's exactly-once semantics are exactly that rather than an exception to it: an idempotent producer removes duplicates from producer retries, and transactions make the consume, process and produce cycle atomic within Kafka. Anything outside that boundary, like charging a card or calling another service, is still ours to make idempotent, and the usual pattern for that is an inbox table keyed by message identifier, written in the same transaction as the business work.

"Why can't you order events by timestamp?"

Because wall clocks on different machines disagree, and they can also jump. NTP keeps them close when it is working and much further apart when it quietly is not, so a write arriving with a later timestamp is not necessarily the one that happened later. That is why last-write-wins by timestamp is a mechanism for silently discarding data rather than a conflict resolution strategy. The alternative is logical ordering: Lamport's happens-before relation, where an event that could have influenced another is ordered before it, implemented as Lamport timestamps for a total order or vector clocks when concurrent writes need to be detected rather than overwritten. In Java the same distinction shows up locally, since `currentTimeMillis` is a wall clock and `nanoTime` is monotonic, and only the second one is safe for measuring elapsed time.

"Explain CAP, and what people get wrong about it."

CAP says that when a network partition happens, a system has to choose between consistency, meaning linearizability, and availability, meaning every request to a non-failing node eventually gets a response. The most common misreading is treating it as pick two of three, as though partition tolerance were optional. It is not, because partitions are a property of networks rather than a design choice, so the real decision is only what happens during one. The second misreading is that the C is the C in ACID, which it is not, since ACID consistency means the declared constraints still hold. I would usually bring up PACELC as the more useful version, because it adds the case that applies the rest of the time: if partitioned, choose availability or consistency, else choose latency or consistency. Partitions are rare, and the latency trade runs constantly.

"How would you choose a partition key?"

By how the data is read and how evenly it spreads, and I would treat it as the hardest decision to reverse, because changing it means moving everything. Range partitioning keeps sorted order so range scans stay cheap, but a sequential key like a timestamp sends every new write to one node while the rest idle. Hash partitioning spreads evenly and destroys that ordering, so a range query has to visit every partition. Consistent hashing is the refinement that matters at scale, because plain hash-modulo-N remaps nearly every key when a node is added, whereas a ring only moves the keys between the new node and its neighbour. The failure I would watch for is a hot partition from a single popular key, which no hashing fixes because it is one key, so that needs an application-level answer like splitting it across several physical keys and merging on read.

"How does a quorum work?"

With N replicas, a write waits for W acknowledgements and a read collects R responses, and when W plus R is greater than N the two sets have to overlap, so the read reaches at least one replica that accepted the newest write. The common setting is three replicas with two for each, and shifting the numbers trades write latency against read latency. Two things I would be careful about: overlap means the read encounters a replica with the newest value, so the client still needs a version to tell which of the returned values that is, and quorum overlap by itself is not linearizability, since concurrent operations can still be observed in different orders without extra machinery. Sloppy quorums weaken it further on purpose, by accepting writes on whichever nodes are reachable and delivering them later through hinted handoff.

"How does Kafka guarantee ordering?"

Within a partition, not across a topic. A topic is split into partitions and messages are ordered inside each one, and the message key decides which partition a message lands in, so all events sharing a key are ordered relative to each other. That makes key choice the ordering decision: keying by order identifier keeps one order's events in sequence while saying nothing about two different orders. In a consumer group each partition goes to at most one consumer, so parallelism is capped by partition count and twenty consumers on ten partitions leaves ten idle. Where the offset is committed decides the delivery semantic, since committing after processing gives at-least-once with redelivery on a crash, and committing before gives at-most-once.

"Why are distributed locks hard?"

Because a process can pause for longer than its lease and never know it. A stop-the-world garbage collection pause or a hypervisor suspending the machine can freeze a JVM for seconds, during which the lease expires and another node legitimately acquires the lock. The first node resumes still believing it holds the lock and writes, so there are two writers and no error anywhere. Timeouts cannot fix it because the paused process cannot tell that time passed. The mechanism that does fix it is a fencing token: the lock service hands out a monotonically increasing number, and the protected resource remembers the highest it has seen and rejects anything lower, so the resumed node's write is refused. The condition there is that the resource has to actually check the token, since one nobody validates is just a number. Where I can, I would rather avoid the lock entirely, with idempotent operations, optimistic concurrency on a version column, or partitioning so one key only ever has one writer.

"A service starts timing out. Why might the whole system go down?"

Because the failure cascades through the callers. The dependency slows, callers hold threads waiting on it, thread pools fill, those callers start failing, their callers retry, and the added load makes the original problem worse. Little's Law is the arithmetic: concurrency is arrival rate times time in the system, so if latency doubles at the same request rate, concurrency doubles and a fixed pool is exhausted. What contains it is bounded queues so overload is rejected rather than turned into growing latency, load shedding so a fraction of requests fail fast and the rest stay healthy, and isolation through circuit breakers and bulkheads so one dependency cannot consume everything. Retries need backoff and jitter for the same reason, since synchronised retries turn a brief outage into a sustained one. The state worth naming is metastable failure, where the system stays down after the trigger is gone because the retry load it generated is enough to keep it down.

"A request across five services is slow. How do you find where?"

Distributed tracing, because logs alone cannot answer it. Each service logs independently, so without a shared identifier there is nothing tying the lines together. A trace identifier names the whole operation and is propagated to every hop, usually in the `traceparent` header from the W3C Trace Context standard, and each operation records a span with a parent, so the trace forms a tree with the time attributed to each node. For an incoming request it is created at the edge, and for a background job or a message consumer it starts wherever that work begins. The way I would use the three signals is that metrics tell me something is wrong, the trace tells me where, and logs tell me what happened at that point, which is why the trace identifier belongs in the logging context so logs and traces can be joined. The practical constraint is cost, since recording everything at volume is expensive, so it gets sampled, and tail-based sampling is preferable when it is affordable because it can keep the slow and failed traces rather than a random selection.

21. Common mistakes

- Treating a remote call like a local one. It can fail in ways a local call cannot, and takes six orders of magnitude longer when it succeeds.
- Retrying without asking whether the operation is idempotent, so a timeout turns one payment into two.
- Ordering events across machines by wall-clock timestamp, and calling the result last-write-wins when it is really data loss.
- Measuring elapsed time with `System.currentTimeMillis()`, which can move backwards, instead of `System.nanoTime()`.
- Describing CAP as picking two of three, or confusing its consistency with ACID consistency.
- Assuming exactly-once delivery exists, rather than building at-least-once delivery with idempotent processing.
- Partitioning on a sequential key such as a timestamp, which sends every new write to one node while the others idle.
- Expecting a hashing scheme to fix a hot key. One popular key is still one key, and spreading it needs an application-level change.
- Using `hash(key) mod N` for partition assignment, then discovering that adding a node remaps nearly every key.
- Adding consumers beyond the partition count and expecting more throughput.
- Committing the offset before processing, which converts redelivery into silent message loss.
- Using an expiring key as a distributed lock where correctness matters, with no fencing token, or with a token the resource never validates.
- Setting a failure detection timeout so short that healthy nodes are evicted under load, which adds load and evicts more of them.
- Using unbounded queues, which turn overload into growing latency and eventually memory exhaustion instead of a clear rejection.

- Sizing a consensus cluster at an even number, which costs a node and tolerates no more failures than the odd number below it.
- Serving liveness and readiness from one endpoint, so a brief dependency outage restarts healthy instances instead of pausing traffic to them.

22. Quick reference

Term	Meaning
Partial failure	Some components fail while others continue, and callers cannot always tell
Two Generals	Two parties over a lossy channel cannot reach certain agreement
Byzantine fault	A node behaving arbitrarily or dishonestly, rather than just stopping
Wall clock	Human time, can jump. Wrong for measuring elapsed time
Monotonic clock	Only moves forward, meaningless across machines. Right for durations
Clock skew	Disagreement between machines' wall clocks, and it grows when sync degrades
Happens-before	<code>a → b</code> if <code>a</code> could have influenced <code>b</code> . Otherwise they are concurrent
Lamport timestamp	A logical counter preserving causality. A total order needs a tie-breaker too
Vector clock	One counter per node, so concurrency can be detected
Linearizability	Operations appear instantaneous and in real-time order
Causal consistency	Causally related operations seen in order, concurrent ones may differ
Eventual consistency	Replicas converge once writes stop. Nothing promised meanwhile
Read your writes	A client always sees its own writes afterwards
CAP	During a partition, choose linearizability or availability
PACELC	Partitioned: A or C. Else: latency or consistency
Replication	Copies of the same data on several nodes
Partitioning	Different data on different nodes. Independent of replication
Consistent hashing	Ring placement, so adding a node moves roughly a <code>1/N</code> share of keys
Hot partition	One key or range taking most of the load, which hashing cannot split
Scatter-gather	A query without the partition key, so every partition is visited
Quorum	<code>W + R > N</code> forces overlap between the write and read sets
Sloppy quorum	Writes accepted on any reachable node, delivered later by hinted handoff

Term	Meaning
FLP	In a fully asynchronous system, deterministic consensus cannot guarantee termination if even one process may crash
$2f + 1$	Crash-fault majority cluster size tolerating f failures. Byzantine needs $3f + 1$
Atomic commit	Every participant must be able to commit, so one failure can block it
Raft	Consensus by terms, elections and majority-committed log entries
Split brain	Two nodes both believing they lead, prevented by requiring a majority
Fencing token	An increasing number the resource validates, making a stale holder harmless
At least once	Retried until acknowledged, so duplicates are possible
Effectively once	At-least-once delivery plus idempotent handling or an atomic side-effect boundary
Outbox and inbox	Atomic publish on the producer, idempotent consumption on the consumer
Consumer lag	Distance between the latest and committed offset. The key queue metric
Backpressure	Signalling upstream to slow down instead of accepting undoable work
Load shedding	Rejecting some requests early to keep the rest healthy
Metastable failure	The system stays down after the trigger, sustained by its own retry load
CRDT	A type whose concurrent updates merge deterministically without coordination
Trace and span	The whole request, and one operation within it

Symptom	Likely cause
A retry creates a duplicate side effect	Non-idempotent handler on at-least-once delivery
Writes silently disappearing under concurrency	Last-write-wins resolution across skewed clocks
A negative or absurd measured duration	Elapsed time measured with a wall clock
Two nodes both acting as leader	Split brain, and no fencing on the protected resource
One node hot while the rest idle	A sequential partition key, or a single popular key
Nearly all data moving when a node is added	Modulo partitioning instead of consistent hashing
A query slow in proportion to cluster size	Scatter-gather, because the partition key was not in the filter
Adding consumers changes nothing	Partition count already matched or exceeded
Messages processed twice after a crash	Offset committed after processing, so a crash before the commit redelivers
Messages missing after a crash	Offsets committed before processing

Symptom	Likely cause
Healthy nodes evicted during load spikes	Failure detection timeout too aggressive
A brief outage becoming a long one	Synchronised retries with no jitter, or metastable retry load
Latency rising steadily until memory runs out	An unbounded queue absorbing overload
A user not seeing their own write	Read served by a lagging replica, breaking read-your-writes
A slow request nobody can attribute	No trace propagation, so each service logged in isolation